



COMPUTER NETWORK • PRACTICAL FILE

STUDENT NAME	UID	BRANCH
Deven	24BCF10060	AIT-CSE
SECTION / GROUP	SEMESTER	SUBJECT
24AIT_KRG-G2	5	Computer Network
SUBJECT CODE	UNIVERSITY	
24CSH-337	Chandigarh University	

8

Implement Secure Connection using TCP

TCP • CONNECTION-ORIENTED

Objective

- ✓ Understand the basic working principles of TCP and its connection-oriented model of communication.
- ✓ Implement a TCP client-server connection using Python socket programming.
- ✓ Establish communication between a **server.py** and a **client.py** script over TCP sockets.
- ✓ Observe connection establishment, data transmission, and connection termination in a live TCP session.

Theory

TCP (Transmission Control Protocol) is a connection-oriented, reliable transport-layer protocol that guarantees ordered, error-checked delivery of data between two hosts on a network. It follows a **client-server model**: the server creates a socket, binds it to an IP address and port, and listens for incoming requests, while the client creates its own socket and connects to the server using that same address and port. Once the connection is accepted, both ends can exchange data freely until either side terminates it.

Reliability in TCP comes from mechanisms such as sequence numbering, acknowledgements, automatic retransmission of lost segments, and flow control — together these make sure data arrives complete, in order, and without duplication. Before any actual data flows, TCP performs a **three-way handshake** to synchronise both sides:

THREE-WAY HANDSHAKE



Practical Steps

- 1 Open the project folder in a code editor such as VS Code, PyCharm, or IntelliJ.
- 2 Create two Python files named **server.py** and **client.py**.

- 3 Paste the server-side code into server.py and the client-side code into client.py.
- 4 Save both files.
- 5 Open two terminals (or a split terminal).
- 6 In Terminal 1, start the server with **python server.py**.
- 7 In Terminal 2, start the client with **python client.py**.
- 8 Exchange a few messages between the client and server CLIs, then type **Over** on the client side to end the session.

Code

```
server.py

import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("127.0.0.1", 5000))  ## bind the server to an IP and port
server_socket.listen(1)
print("Server started")
print("Waiting for a client...")

client_socket, client_address = server_socket.accept()
print("Client connected:", client_address)

while True:
    message = client_socket.recv(1024).decode()
    print("Client:", message)
    if message == "Over":
        break
    reply = input("Enter reply: ")
    client_socket.send(reply.encode())

client_socket.close()
server_socket.close()
print("Connection closed")
```

```
client.py

import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000))
print("Connected to server")

while True:
    message = input("Enter message: ")
    client_socket.send(message.encode())  ## encode(): string -> bytes
    if message == "Over":
        break
    reply = client_socket.recv(1024).decode()
    print("Server:", reply)

client_socket.close()
print("Connection closed")
```

● Result

```
client.py - Terminal 2

PS D:\Deven\CN_Lab\TCP_Experiment> python client.py
Connected to server
Enter message: Hi, this is Deven (24BCF10060)
Server: Hey Deven, handshake looks good on this end
Enter message: Great, testing a normal data exchange now
Server: Received it fine, sending an ack-style reply
Enter message: Perfect, closing the session
Server: Sounds good, talk soon
Enter message: Over
Connection closed
```

```
server.py - Terminal 1

PS D:\Deven\CN_Lab\TCP_Experiment> python server.py
Server started
Waiting for a client...
Client connected: ('127.0.0.1', 51422)
Client: Hi, this is Deven (24BCF10060)
Enter reply: Hey Deven, handshake looks good on this end
Client: Great, testing a normal data exchange now
Enter reply: Received it fine, sending an ack-style reply
Client: Perfect, closing the session
Enter reply: Sounds good, talk soon
Client: Over
Connection closed
```

● Learning Outcome

- ★ Understood the working and characteristics of TCP, including connection establishment, reliable data transfer, and graceful connection termination.
- ★ Built a working TCP client-server application using Python socket programming.
- ★ Analysed how a connection is established and how data is exchanged between a client and a server over TCP.